SE465/ECE653
Software Testing and Quality Assurance
Assignment 3, version 1*

Patrick Lam
Release Date: March 16, 2026

Due: 11:59 PM, Monday, April 6, 2026
Submit: via git.uwaterloo.ca and Crowdmark

Getting set up

We will create a copy of the starter repo for you in your `git.uwaterloo.ca` account. You need to log in to `git.uwaterloo.ca` for that to work.

I expect each of you to do the assignment independently. As stated in the course outline, you can ask questions of generative AI, but you cannot submit text or code that comes from GenAI. I will follow UW's Policy 71 for all cases of plagiarism.

Submission instructions:

Commit **and push** your modifications back to your fork on `git.uwaterloo.ca`. It's git, so you can submit multiple times. After submission, **please make a fresh clone of your submission to make sure you have uploaded all necessary files**.

Submission summary

Here's what you need to submit in your fork of the repo. Be sure to commit and push your changes back to `git.uwaterloo.ca`.

- Q1: `kani-slow-max/src/lib.rs`
- Q2: `kani-failures/src/main.rs` and `kani-failures-discussion.md`
- Q3: `dafny/FindRecursive.dfy`
- Q4: `dafny/Sqrt.dfy`; `dafny/CalcTerm.dfy`; `dafny/SlowMax.dfy`
- Q5: `dafny/StringRevArray.dfy`
- Q6: `dafny/pancakesort/findmax.dfy`; `dafny/pancakesort/flip.dfy`; `dafny/pancakesort/PancakeSort.dfy`; `dafny/quicksort/part.dfy`; (bonus: `dafny/quicksort/QuickSort.dfy`.

*version 1: initial release

For Questions 1 and 2 you'll need to install Rust and Kani.

- Rust installation instructions: `https://doc.rust-lang.org/book/ch01-01-installation.html`
- Kani installation instructions: `https://model-checking.github.io/kani/install-guide.html`

You can install Kani 0.57.0 on the `eceubuntu` machines in your personal account.

```
cargo install --locked kani-verifier --version 0.57.0 -Znext-lockfile-bump
```

Question 1: Kani proof harness (10 points)

In the `kani-slow-max` directory you will find `src/lib.rs`, which contains a Rust function `slow_max()`. (This function comes up again later, but implemented in Dafny). There is also a normal Rust test `test_slow_max()`.

Write Kani proof harnesses for `slow_max()`. There should be a harness where parameter `a` to `slow_max()` is larger, and one where `b` is larger.

I've provided a skeleton for `proof_harness_slow_max_a`. Since Kani does bounded model checking, you have to set an unwinding limit. You may also tell Kani to assume bounds for `a` and `b`.

Add a comment in `lib.rs` saying what you can conclude from your proof harnesses.

Question 2: Kani and program errors (10 points)

In the `kani-failures` directory of your A3 repo, you will find some skeleton proof harnesses. Write code so that `cargo kani` will make Kani (and not `rustc`) report a divide by zero error; an array out of bounds error; a unwrap-None error; and one more kind of error. In `kani-failures-discussion.md`, include minimal excerpts from `cargo kani`'s output showing the failure, and describe the additional error that you found using Kani. The three errors that I've put skeletons for are worth 2 points each, while the novel error is 4 points.

For the remaining questions, you'll need to use Dafny.

- Dafny installation instructions: `https://dafny.org/latest/Installation`

Question 3: Dafny FindRecursive Contract (5 points)

Here is a slide from the SE212 notes, which I've included with permission. In that class, you manually proved the correctness of this recursive code.

Pre: $\exists k \bullet 0 \leq k \wedge k \leq i \wedge B[k] = c$

```
fun find(i) {
    assert(Pre);
    if (B[i]=c) then {


        ret := i;

    } else {


        ret := find(i-1);

    }
    assert( B[ret] = c)
    return ret;
}
```


I've translated the code into Dafny, and you'll find it in `dafny/FindRecursive.dfy`. Your job is to add the correct `requires` and `ensures` clauses for this method. With these clauses, the method must pass Dafny verification. You don't need to make any changes to the implementation for this question.

```
method Find(B:array<int>, target:int, i:int) returns (ind:int)
{
  if B[i] = target {
    ind := i;
  } else {
    ind := Find(B, target, i-1);
  }
}
```

Question 4: Small Dafny Problems (15 points)

For this question, you will have to verify three methods in Dafny.

All files for this question are in the `dafny` folder of the repository.

(a) **`Sqrt.dfy`**: (5 points) The file contains two methods

```
SqrtRoot1(N: nat) returns (r:nat)
```

and

```
SqrtRoot2(N: nat) returns (r:nat)
```

which compute the largest r such that $r \in \mathbb{Z} \wedge r \leq \sqrt{N}$.

Provide the missing invaraints so that the methods verify (2.5 points for each method).

(b) **`CalcTerm.dfy`**: (5 points) The file contains a method

```
CalcTerm(m:int, n:nat) returns (res:int)
```

The method computes $5 \times m - 3 \times n$ using a loop and operations to add 5, and subtract 1 and 3.

Provide the missing invariants so that the method verifies.

(c) **`SlowMax.dfy`**: (5 points) The file contains a method

```
SlowMax(a:nat, b:nat) returns (z:nat)
```

that computes $z = \max(a, b)$ via a series of increments and decrements.

Provide the missing invariants so that the method verifies.

Question 5: String Reversal (10 points)

For this question, you will have to provide one predicate and verify one methods in Dafny.

All files for this question are in the `dafny` folder of the repository.

(a) **`StringRevArray`**: (10 points) The file contains a skeleton of a predicate `isReversed` and a method `StringReverse`.

Provide the definition of `isReversed` and the missing invariants so that the method verifies.

Question 6: Sorting (30 points)

For this question, you will have to verify two implementations of sorting methods in Dafny.

All files for this question are also in the `dafny` folder of your repository.

(a) (15 points) Pancake sorting[1] is a sorting technique that uses only a flip operation, in which elements of an array are reversed (flipped) at some position. The subdirectory `pancakesort` contains an implementation of the pancake sorting algorithm in Dafny.

- Specify and verify method `findMax` in `findmax.dfy`
- Specify and verify method `flip` in `flip.dfy`
- Specify and verify method `pancakeSort` in `PancakeSort.dfy`

(b) (15 points) Quicksort[2] is an efficient sorting technique developed by the late Tony Hoare. The subdirectory `quicksort` contains an implementation of quick sort in Dafny.

- Write a loop invariant to complete the verification of `partition` method in `part.dfy`
- *(Bonus question, max 15 points)* Complete and verify the implementation of `qsort` method in `QuickSort.dfy`. You might find Chapter 6 of *The Calculus of Computation*[3] useful for this problem.

[1] https://en.wikipedia.org/wiki/Pancake_sorting
[2] https://en.wikipedia.org/wiki/Quicksort
[3] https://link.springer.com/book/10.1007/978-3-540-74113-8